

Security Information and Event Management System

A Project Report Submitted in Partial Fulfillment of the
Requirements for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

of

ASSAM SCIENCE AND TECHNOLOGY UNIVERSITY



Submitted by

AVIROOP BEZBARUAH 210710007009

INDRAJIT MANDAL 210710007017

SUBHAM BANIK 210710007050

Under the guidance of

MR. RAJIB CHAKRABARTTY

Assistant Professor at JEC

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
JORHAT ENGINEERING COLLEGE, JORHAT, ASSAM**

Year of submission : 2025

Candidate's Declaration

We hereby declare that the work entitled “**Security Information and Event Management System**” is a project work submitted in fulfillment of the requirement for the Final Year Project of **Bachelor of Technology in Computer Science and Engineering** carried out during the academic 8th semester from January to May of the year 2025.

This Project is a record of bonafide work carried out by our group under the supervision of **Mr. Rajib Chakrabartty**, Assistant Professor, Department of Computer Science and Engineering, Jorhat Engineering College.

We further declare that the work reported in this project has not been and will not be submitted, either in part or in full, for the award of any other programme in this organization or any other organization or institute.

Roll No.	Name of the candidate	Signature
210710007009	Aviroop Bezbaruah	
210710007017	Indrajit Mandal	
210710007050	Subham Banik	

Approval Sheet

The project titled “**Security Information and Event Management System**” is approved for the degree **Bachelor of Technology in Computer Science and Engineering** under **Assam Science and Technology University**

Examiner:

.....

Guide:

.....

Head of Department:

.....

Date and Place:

.....

.....

Certificate Of Guidance

This is to certify that the project titled “**Security Information and Event Management System**” is submitted by **Aviroop Bezbaruah, Indrajit Mandal, Subham Banik** in partial fulfillment of the requirements for the award of the degree in **Bachelor of Technology in Computer Science and Engineering** of the department of Computer Science and Engineering from **Jorhat Engineering College** under **Assam Science and Technology University** has been carried out under my guidance.

The contents of the report have not been submitted in part or full to any other university for the award of any other degree or diploma.

Date:.....

Place:.....

Signature of the Guide:

.....

Mr. Rajib Chakrabartty

Assistant Professor,

Department of Computer Science and Engineering

Jorhat Engineering College, Jorhat, Assam

Certificate Of Examination

This is to certify that **Aviroop Bezbaruah, Indrajit Mandal, Subham Banik** of B.Tech 8th semester of the department of Computer Science and Engineering from Jorhat Engineering College have completed the project titled **“Security Information and Event Management system”** under the guidance of Mr. Rajib Chakrabartty, Assistant Professor, Department of Computer Science and Engineering, Jorhat Engineering College, Assam. We have examined this report and hereby record and approve for it as a successful project work.

Signature of guide:

.....

Signature of HOD:

.....

Signature of External Examiner:

.....

Acknowledgements

We extend our deepest gratitude to the following individuals whose unwavering support, guidance, and encouragement have been indispensable throughout the entirety of our project journey. Their expertise, dedication, and generosity of time have significantly contributed to the successful completion of our endeavors:

Mr. Rajib Chakrabartty, Assistant Professor, Department of Computer Science and Engineering:

We express our heartfelt gratitude to our guide, Mr. Rajib Chakrabartty, for his invaluable guidance, support, and encouragement throughout the course of this project. His expertise and insights have been instrumental in helping us navigate challenges and refine our approach.

Dr. Diganta Baishya, Head of the Department, Department of Computer Science and Engineering:

We extend our sincere appreciation to Mr. Diganta Baishya for his invaluable support and for providing us with the necessary resources and facilities within the department.

Dr. Rupam Baruah, Principal, Jorhat Engineering College:

Dr. Rupam Baruah's unwavering guidance and patient mentorship, have been instrumental in steering us through the intricate challenges encountered during this project.

Additionally, we express our gratitude to the esteemed faculty and staff members of the Department of Computer Science and Engineering for their unwavering assistance, valuable insights, and encouragement throughout our journey. Their collective expertise and dedication have enriched our learning experience and have played a pivotal role in shaping the outcome.

Furthermore, we would like to acknowledge the unwavering support and understanding extended by our parents, whose encouragement and sacrifices have been a constant source of strength and motivation. We are also grateful to our friends for their encouragement, camaraderie, and unwavering belief in our abilities.

Together, their support and guidance have facilitated the successful completion of this project and have also enriched our academic and personal growth. We are grateful for their contributions and unwavering commitment to our success.

Abstract

This project presents the development of an intelligent Security Information and Event Management (SIEM)[1, 2] system designed to classify system logs into Anomalous or Normal categories. SIEM systems play a vital role in modern cybersecurity by analyzing logs in real-time and identifying potential threats. Our initial approach used LogBERT[3, 4], a BERT-based transformer model, but we encountered major limitations due to its high computational requirements and the complexity of domain-specific fine-tuning.

Following a review of industry solutions like CrowdSec[5, 6, 7], we transitioned to a rule-based method involving log parsing and decision-tree-like logic. This approach offered high interpretability and low resource demand while effectively identifying anomalous patterns based on structured rules and keyword matches.

To improve context-awareness, we enhanced the model using short-Time To Live queues. This enabled our system to detect behavioral patterns over time without relying on heavyweight machine learning infrastructure.

This report outlines the system architecture, key challenges, and the rationale behind our methodology shift. We also describe future goals, including expanding rule sets, testing on varied datasets, and deploying the system as a functional SIEM prototype.

As students, we faced multiple constraints such as limited access to high-performance computing resources, the scarcity of comprehensive and labeled log datasets, and the lack of open documentation or academic guidance in building SIEM systems, which are often proprietary and industry-gatekept. Despite these challenges, the project illustrates how adaptive thinking and the use of industry-informed heuristics can lead to the development of lightweight, practical security tools.

Keywords: SIEM, Log Analysis, Cybersecurity, Log Parsing, Rule-Based Detection, Anomaly Detection

Table of Contents

1	Introduction	1
2	Literature Review	2
2.1	Introduction to Log Analysis	2
2.2	Methods and Techniques in Log Analysis	2
2.3	Machine Learning in Log Analysis	3
2.4	Types of Machine Learning Methods in Log Analysis	4
2.5	Natural Language Processing (NLP) in Log Analysis	6
2.6	LogBERT: A BERT-Based Model for Log Analysis	6
2.7	Challenges in Log Analysis using logBERT	8
2.8	Keyword-Based Parsing in Log Analysis	9
2.9	Rule-Based and Decision-Based Methods in Log Analysis . .	9
2.10	Security Information and Event Management Systems	12
2.11	Technology Stacks in SIEM Systems	14
2.12	Comparative Studies and Learning Outcomes	15
3	Methodology	17
3.1	Initial Approach: BERT-based ML Model	17
3.2	Revised Approach: Log Parsing and Classification	18
4	Implementation Steps and Details	20
4.1	Synthetic Log Generation Methodology	20
4.2	Implemented Rule-Based Methods	22
4.3	Short-Term Memory-Based Context Learning	24
4.4	Technology Stack	28

5	System Design and Workflow	30
5.1	Linux Server Client	30
5.2	Backend System (FastAPI)	30
5.3	Frontend System (ReactJS)	31
5.4	Database Stack	32
5.5	Communication Flow	32
5.6	Workflow Summary	32
5.7	Flowchart of the System Design	34
6	Future Prospects	35
7	Conclusion	37
	<i>Bibliography</i>	38

List of Figures

4.1	Score vs IP for batch 1	25
4.2	Score vs IP for batch 2	25
4.3	Score vs IP for batch 3	26
4.4	Score vs IP for batch 4	26
4.5	Admin Interface of the Dashboard	28
4.6	Basic structure of our database	29
5.1	System Design and Workflow	34

Chapter 1

Introduction

In today's rapidly evolving digital landscape, cybersecurity has become a critical concern for organizations worldwide. Security Information and Event Management (SIEM)[8, 9, 10] systems play a pivotal role in safeguarding networks by providing real-time analysis of security alerts and logs generated by applications, servers, and network devices. These logs are essential for identifying potential threats, anomalies, and breaches. However, the sheer volume and complexity of log data pose significant challenges for effective analysis and classification.

This project aims to address the challenge of efficiently classifying log data into two categories: Anomalous or Normal. Accurate log classification is vital for detecting security threats promptly and minimizing false positives, which can otherwise overwhelm security teams and hinder their ability to respond effectively.

Initially, we explored the application of advanced machine learning techniques using BERT (Bidirectional Encoder Representations from Transformers)[11, 12, 13] to classify logs. Although BERT has shown remarkable success in natural language processing, its implementation in our project proved to be overly complex, given the computational requirements and the intricacies of fine-tuning it for log analysis. This realization led us to pivot to a more practical and scalable approach.

Through a detailed study of industry practices, particularly the methodologies employed by CrowdSec, we identified an alternative solution. CrowdSec's documentation highlighted the use of keyword-based log parsing and decision tree classifiers, which align well with our project's scope and resources. Inspired by this, we developed a Log Parser that extracts key features from logs and employs a Decision Tree-based classifier to determine whether a log entry represents anomalous or normal behavior.

The objective of this report is to provide an overview of the project's progress, including the challenges faced, the pivot in methodology, and the development of the log parser and classifier, to create an accessible and efficient solution for log classification that can contribute to improving cybersecurity systems.

Chapter 2

Literature Review

2.1 Introduction to Log Analysis

Log analysis refers to the process of examining and interpreting log data generated by various systems, applications, and networks. These logs serve as a crucial source of information, containing records of events such as system activities, error messages, performance metrics, and user interactions. Log analysis is fundamental for tasks such as system monitoring, troubleshooting, performance optimization, and security management. The increasing complexity and volume of log data, driven by the adoption of distributed systems, microservices, and cloud computing, has made manual analysis impractical. Consequently, sophisticated techniques and automated tools are required to efficiently process and extract meaningful insights from log files.

2.2 Methods and Techniques in Log Analysis

Several techniques have been developed to address the challenges of analyzing diverse and voluminous log data. A key step in log analysis is log parsing, which involves transforming unstructured or semistructured log data into a structured format suitable for analysis. Traditionally, this was achieved using manually created templates or regular expressions, which are simple but lack scalability and flexibility. Modern approaches leverage machine learning models, such as LSTM-based parsers or attention mechanisms, to dynamically learn parsing rules. These techniques are particularly effective for handling heterogeneous log formats generated by different systems.

Anomaly detection is another critical aspect of log analysis. Specifically, it focuses on identifying unusual patterns or deviations from normal behavior within the logs. Statistical techniques, such as z-score analysis or moving averages, compare current system behavior with historical data to pinpoint anomalies. In addition, unsupervised learning algorithms, such as clustering, can group similar events and identify outliers as potential anomalies. Hybrid approaches combine rule-based systems with machine learning models to provide robust solutions capable of adapting to dynamic environments.

Visualization plays a significant role in log analysis by converting raw data into

easily interpretable graphical representations. Tools such as Kibana, Grafana, and Splunk offer dashboards that allow users to visualize log data using graphs, graphs, and heatmaps. Advanced visualization techniques enable users to interact with these representations, drill down into specific timeframes, and gain a deeper understanding of system behavior.

Another essential method in log analysis is correlation analysis, which identifies relationships between events across multiple logs. This technique is particularly useful in distributed systems, where events from different components must be correlated to identify issues. Traditional methods involve sequence alignment and cross-log correlation, while modern machine learning models can uncover temporal patterns and causal links, making root cause analysis more efficient.

Pattern mining is used to discover frequent or recurring sequences in log data. For example, identifying patterns of standard operations or common error sequences helps to predict and prevent potential problems. Algorithms like Apriori and PrefixSpan have been adapted to mine sequential patterns from log datasets. These insights are often incorporated into predictive models that help organizations proactively address recurring problems.

2.3 Machine Learning in Log Analysis

Machine learning has significantly advanced the field of log analysis by automating many manual and time-consuming processes. Supervised learning is often applied to classify log events or predict specific outcomes based on labeled datasets. For instance, models like Random Forest, Support Vector Machines (SVM), and Gradient Boosted Trees are widely used to classify error types or predict hardware failures. These models excel at identifying specific patterns in the logs associated with known outcomes.

Unsupervised learning, on the other hand, is particularly effective for exploratory analysis of log data. This approach is used when labeled datasets are unavailable, which is often the case with large-scale systems. Clustering algorithms like k-means and DBSCAN group similar log events to reveal hidden patterns, while dimensionality reduction techniques such as Principal Component Analysis (PCA) and t-SNE are used to reduce the complexity of high-dimensional log data for easier visualization and interpretation.

Deep learning models have also been widely adopted in log analysis due to their ability to handle sequential and textual data effectively. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are particularly well-suited for analyzing log sequences and detecting anomalies over time. Transformer-based models, which use self-attention mechanisms, are increasingly being applied to model long-range dependencies in log data, offering improved performance in anomaly detection and event prediction.

Natural Language Processing (NLP) techniques have proven useful for logs that contain textual data, such as error messages or user activity descriptions. Text embeddings, such as Word2Vec or BERT, transform log messages into vector representations that capture semantic meaning. These embeddings are used to

cluster similar logs, classify log events, or uncover dominant themes using topic modeling techniques like Latent Dirichlet Allocation (LDA).

Reinforcement learning is another emerging approach in log analysis. It is often applied to optimize log parsing and analysis workflows. By incorporating feedback from system administrators, reinforcement learning agents learn to prioritize log events, refine parsing rules, and improve decision-making processes over time.

2.4 Types of Machine Learning Methods in Log Analysis

Machine learning (ML) has revolutionized the field of log analysis by automating the detection of anomalies, classification of events, and extraction of patterns from large-scale log data. This subsection explores the key machine learning methods used in log analysis, categorized into supervised, unsupervised, and semi-supervised learning techniques, with detailed mathematical formulations and their practical relevance.

1. Supervised Learning Supervised learning techniques require labeled log data to train models that predict specific outputs based on given inputs. These methods are widely used for tasks such as anomaly detection, log classification, and event prediction.

Consider a labeled dataset of logs $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, where x_i represents a log entry or sequence, and y_i is the corresponding label (e.g., anomalous or normal). The objective of supervised learning is to find a function $f : X \rightarrow Y$ that minimizes the prediction error:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i),$$

where ℓ is a loss function (e.g., cross-entropy loss for classification tasks).

Common supervised learning algorithms in log analysis include:

- **Random Forest Classifiers (RFC):** Ensembles of decision trees that aggregate predictions for robust classification of logs.
- **Support Vector Machines (SVM):** Effective for binary classification of logs into normal and anomalous categories.
- **Deep Learning Models:** Neural networks, including Long Short-Term Memory (LSTM) and Recurrent Neural Networks (RNNs), are commonly used to capture sequential and temporal patterns in log data for improved anomaly detection.

2. Unsupervised Learning Unsupervised learning is ideal for scenarios where labeled log data is unavailable. These methods aim to uncover hidden structures or anomalies within the data.

Let $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$ represent the set of unlabeled logs. Unsupervised methods aim to optimize an objective function J based on the structure of $\mathcal{X}$. Examples include:

- **Clustering:** Grouping similar logs together based on features. For instance, k -means minimizes the intra-cluster variance:

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2,$$

where C_j is the j -th cluster, and μ_j is its centroid.

- **Dimensionality Reduction:** Techniques like Principal Component Analysis (PCA) reduce log data dimensions, retaining critical patterns for analysis.

Unsupervised anomaly detection algorithms, such as Isolation Forest and Autoencoders, are particularly useful for detecting rare and unknown log patterns.

3. Semi-Supervised Learning Semi-supervised learning bridges the gap between supervised and unsupervised methods by utilizing a small set of labeled logs alongside a larger pool of unlabeled data. These methods leverage the labeled data to guide the learning process on the unlabeled data.

Given labeled data $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^m$ and unlabeled data $\mathcal{U} = \{x_j\}_{j=1}^n$, the goal is to minimize the combined loss:

$$\mathcal{L}_{\text{semi}} = \mathcal{L}_{\text{supervised}} + \lambda \mathcal{L}_{\text{unsupervised}},$$

where λ controls the contribution of the unsupervised loss.

Techniques like self-training, co-training, and graph-based methods are commonly applied. For example, graph neural networks (GNNs) model the relationships between logs based on their semantic or temporal similarities.

4. Reinforcement Learning Reinforcement learning (RL) is an emerging area in log analysis, where the system learns to optimize a reward signal based on its actions. For example, RL can be used to dynamically adapt monitoring rules or to automate responses to log-detected anomalies.

The RL framework involves:

- **State (s):** Representation of the current log context.
- **Action (a):** Decision made by the model (e.g., classify as anomalous or normal).
- **Reward (r):** Feedback received for the action.

The goal is to find a policy $\pi(a|s)$ that maximizes the expected cumulative reward:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right],$$

where γ is the discount factor.

Machine learning methods, encompassing supervised, unsupervised, semi-supervised, and reinforcement learning, offer diverse tools for effective log analysis. Each method has its strengths and limitations, making the choice of method dependent on the availability of labeled data, the nature of logs, and the specific objectives of the analysis. By leveraging these methods, systems can achieve automated, accurate, and scalable log analysis.

2.5 Natural Language Processing (NLP) in Log Analysis

Research in the application of Natural Language Processing (NLP) for log analysis has grown significantly. Techniques such as BERT (Bidirectional Encoder Representations from Transformers) have demonstrated their potential in understanding and classifying text-based data. A derived version of BERT is LogBERT[14] which shows that BERT can be fine-tuned for domain-specific tasks, including log classification. However, these methods are computationally intensive and require large datasets for effective training. We initially attempted to implement a BERT-based ML model for log classification. Despite its theoretical promise, practical challenges such as the need for extensive computational resources, fine-tuning complexity, and overfitting in small datasets highlighted the limitations of this approach for our project.

2.6 LogBERT: A BERT-Based Model for Log Analysis

LogBERT is a specialized adaptation of the Bidirectional Encoder Representations from Transformers (BERT) model, tailored for analyzing system logs. It leverages BERT’s deep bidirectional learning capabilities to understand log sequences, detect anomalies, and classify events. Below, we explore the underlying principles, mathematical foundations, and key steps in implementing LogBERT.

1. Tokenization of Log Data The implementation begins by converting raw log data into a structured format suitable for input to the model. Each log message is treated as a sequence of tokens.

The tokenization process can be described mathematically as:

$$\mathbf{T} = [t_1, t_2, \dots, t_n],$$

where $\mathbf{T}$ represents the tokenized sequence, and t_i is the i -th token derived from the raw log data. Subword tokenization methods, such as Byte Pair Encoding (BPE), are used to split log messages into meaningful sub-tokens.

For instance:

Log Message: "Error 404: Page Not Found"

is tokenized into:

$$\mathbf{T} = ["Error", "404", ":", "Page", "Not", "Found"]$$

2. Input Representation Each token is represented as an embedding vector. The input to LogBERT combines:

- **Token Embeddings:** Represent individual tokens.
- **Position Embeddings:** Indicate the position of each token in the sequence.
- **Segment Embeddings:** Used for distinguishing different parts of the input (e.g., log types or events).

The combined input representation is:

$$\mathbf{E}_i = \mathbf{T}_i + \mathbf{P}_i + \mathbf{S}_i,$$

where $\mathbf{E}_i$ is the embedding for the i -th token, $\mathbf{T}_i$ is the token embedding, $\mathbf{P}_i$ is the position embedding, and $\mathbf{S}_i$ is the segment embedding.

3. Transformer Encoder LogBERT utilizes the Transformer architecture to process the input sequence. The Transformer encoder applies self-attention to capture dependencies between tokens across the entire sequence.

The self-attention mechanism is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where: - Q , K , and V are the query, key, and value matrices derived from the input embeddings. - d_k is the dimension of the key vectors, used for scaling.

The multi-head self-attention layer enhances the model's ability to focus on different aspects of the log sequence:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O,$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, and W^Q, W^K, W^V, W^O are learned projection matrices.

4. Pretraining Objectives LogBERT is pretrained using unsupervised learning objectives adapted for log data:

- **Masked Token Prediction (MLM):** A percentage of tokens in the input sequence are masked, and the model learns to predict them:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in M} \log P(t_i | \mathbf{T}),$$

where M is the set of masked positions.

- **Next Log Prediction (NLP):** The model predicts whether a given log sequence logically follows another, enhancing its ability to capture sequential relationships:

$$\mathcal{L}_{\text{NLP}} = - \sum [y \log \hat{y} + (1 - y) \log(1 - \hat{y})],$$

where y is the ground truth label (1 for true, 0 for false), and $\hat{y}$ is the

model’s prediction.

5. Fine-Tuning for Log Analysis Tasks After pretraining, LogBERT is fine-tuned for specific tasks such as anomaly detection and event classification. The fine-tuning process adapts the model to labeled datasets by optimizing task-specific loss functions, such as cross-entropy for classification:

$$\mathcal{L}_{\text{classification}} = - \sum_{i=1}^C y_i \log \hat{y}_i,$$

where C is the number of classes, y_i is the true label, and $\hat{y}_i$ is the predicted probability.

6. Implementation Pipeline The implementation pipeline for LogBERT involves: 1. Preprocessing raw logs (tokenization, normalization). 2. Pretraining the model on large-scale unlabeled logs using MLM and NLP objectives. 3. Fine-tuning the model on task-specific datasets. 4. Evaluating the model using metrics like precision, recall, and F1-score.

7. Advantages of LogBERT LogBERT outperforms traditional log analysis methods due to its ability to: - Capture contextual and sequential information. - Generalize to unseen patterns and anomalies. - Adapt to various log formats and domains through transfer learning.

This makes LogBERT a powerful tool for modern log analysis challenges.

2.7 Challenges in Log Analysis using logBERT

This literature highlights several challenges in log analysis, including managing unstructured data, processing large-scale logs, and reducing false positives. These challenges became particularly clear during our early experiments with BERT. The preprocessing and tokenization steps required for log data proved to be complex and time-consuming. Implementing LogBERT for log analysis presents several challenges for students, primarily due to its computational and resource-intensive nature. LogBERT’s reliance on high-performance GPUs (Graphics Processing Units) or TPUs (Tensor Processing Units) and its quadratic memory complexity for long sequences often exceeds the capabilities of standard hardware. Preprocessing raw logs, which are typically unstructured and noisy, adds further complexity, requiring tokenization, normalization, and handling of imbalanced datasets. The scarcity of labeled log data, coupled with the need for domain-specific customization, makes fine-tuning particularly challenging, as logs vary widely across industries and systems. Additionally, we struggled with understanding the intricacies of transformer architectures, hyperparameter tuning, and the implementation of pretraining objectives like Masked Language Modeling (MLM). While LogBERT excels at detecting patterns and anomalies, its black-box nature makes interpretability difficult, especially in academic contexts where understanding model behavior is critical. Deployment in real-time systems introduces latency issues and integration difficulties, further complicating its practical

use. Financial barriers, such as the cost of cloud resources for training and deployment, and ethical considerations, such as managing sensitive log data and mitigating biases in the training process, add to the challenges. Despite these hurdles, with access to smaller datasets, prebuilt models, and simplified tasks, students can gradually gain the skills necessary to leverage LogBERT effectively for advanced log analysis.

To address these difficulties, simpler methods like keyword-based parsing and classifier trees have proven effective. These approaches streamline the process by focusing on structured data and applying rule-based classification, making log analysis more manageable and efficient.

2.8 Keyword-Based Parsing in Log Analysis

Keyword-based log analysis has been widely adopted in the cybersecurity industry due to its simplicity and efficiency. CrowdSec, an open-source collaborative cybersecurity platform, provides documentation on using log parsers and rule-based systems for anomaly detection. Their approach involves identifying specific keywords or patterns in logs that correlate with anomalous or normal behavior, which can then be used to build decision trees for classification. This methodology inspired a pivot in our project. By leveraging a parser to extract relevant keywords and features from logs, we could reduce complexity while maintaining effectiveness.

2.9 Rule-Based and Decision-Based Methods in Log Analysis

Rule-based and decision-based approaches are foundational in log analysis, especially within the domain of Security Information and Event Management (SIEM). These methods rely on deterministic logic: pre-defined patterns, conditions, or decision rules to detect security-relevant anomalies. Compared to complex statistical or machine learning methods, they offer clarity, low latency, and resource efficiency – making them particularly suitable for environments with real-time constraints or strict auditability requirements.

Core Principles

In rule-based systems, detection mechanisms operate through a series of structured conditions crafted to match specific log patterns. These include:

- **Keyword Matching:** This involves identifying log entries containing specific strings or phrases. For example, entries that include "failed password" or "unauthorized access" might immediately raise alerts. It is one of the simplest forms of log parsing and is effective when log formats and threat expressions are consistent.
- **Regular Expressions:** Regular expressions (regex) allow for more flexible and powerful pattern matching, accommodating variations in log formats. This approach is especially useful for detecting attack signatures or com-

mand sequences that may vary slightly across instances but still represent the same threat.

- **Threshold Triggers:** This method involves counting the frequency of specific events within a predefined time frame. For instance, if more than five login failures occur within a minute, a brute-force attempt might be inferred. Thresholds can be static or dynamically adjusted based on historical data.
- **Correlation Rules:** These involve linking multiple discrete log events across time or systems to identify complex attack patterns. An example might be a successful login immediately followed by privilege escalation or abnormal access to sensitive directories.

Decision-based systems, such as those using decision trees or rule flowcharts, structure detection logic into branching paths. These branches are determined by evaluating specific features in the logs – such as IP geolocation, severity levels, or user privileges – and lead to deterministic outcomes without the need for training data.

Advanced Techniques and Enhancements

Modern implementations of rule-based systems incorporate various advanced features that extend their analytical capability:

- **Temporal Rules:** These are rules that consider the timing of log events. For example, detecting a low-frequency brute-force attack might require observing a pattern over several hours or days, which typical threshold rules might miss.
- **Hierarchical Rules:** These rules are structured in layers, where low-level rules detect simple patterns and feed into higher-level rules representing broader scenarios. This modular design improves maintainability and allows for complex detections without redundancy.
- **Adaptive Rule Tuning:** Rule thresholds and parameters are updated based on observed log data. This dynamic adjustment reduces false positives and adapts to changing baseline behaviors, making the system more resilient to legitimate variability in activity.
- **Stateful Pattern Matching:** This technique involves keeping track of past log events and applying logic based on event sequences rather than isolated occurrences. It is particularly valuable for detecting multi-stage attacks or behavioral anomalies that evolve over time.

Hybrid Systems with Rule-Based Layers

Hybrid detection architectures often leverage rule-based systems as the first layer of defense. In these systems:

- The rule-based layer performs initial filtering and triaging, eliminating clearly benign or known malicious patterns efficiently.

- More computationally expensive machine learning models are invoked only on events that pass this initial filter, thereby reducing processing load and improving scalability.
- Insights from machine learning models can feed back into the rule base, refining existing rules or suggesting new ones based on behavioral anomalies identified during runtime.

This layered approach combines the interpretability and speed of rule-based systems with the adaptability of learning-based methods.

Industry Usage and Notable Tools

Rule-based methods are extensively used in both open-source and commercial security platforms:

- **CrowdSec:** This open-source system uses YAML-defined scenarios that evaluate log entries based on frequency and pattern conditions [5]. It incorporates community-shared threat intelligence, enhancing the accuracy and relevance of its rules.
- **OSSEC:** A host-based intrusion detection system (HIDS) that supports customizable rules for parsing and correlating events [15].
- **Sysmon with Sigma Rules:** Sysmon provides detailed system event logs which can be analyzed using Sigma [16], a generic and portable rule format translatable into query languages for SIEM platforms like Splunk and Elasticsearch.
- **Snort and Suricata:** These network intrusion detection systems (NIDS) use rule engines to scan packet-level data in real time [17, 18]. Their effectiveness in detecting known attack signatures makes them suitable for high-performance network monitoring.
- **Falco:** Tailored for containerized environments like Kubernetes, Falco monitors system calls and applies behavior-based rules to detect suspicious activity [19], offering protection in dynamic cloud-native infrastructures.

Advantages

- **High Interpretability:** Every decision or alert raised by a rule-based system can be directly traced to a specific rule, aiding in debugging and auditability.
- **Low Resource Requirements:** Unlike machine learning models, rule-based systems do not require data training, feature extraction, or continuous updates, making them lightweight and efficient.
- **Real-time Performance:** Because rules are applied as logs are ingested, these systems can respond to threats with minimal delay.
- **Regulatory Compliance:** Rule-based systems facilitate documentation and justification of detection logic, which is essential for regulatory audits and certifications.

Limitations

- **Limited Novelty Detection:** Static rules cannot identify new or evolving threats that do not match known patterns.
- **High Maintenance Overhead:** As the number of rules grows, managing, prioritizing, and debugging them becomes increasingly complex.
- **False Positives:** Rigid conditions may not account for contextual factors, leading to benign events being flagged as anomalies.
- **Scalability Issues:** In large-scale deployments, ensuring consistent rule application across diverse log sources and formats poses a significant challenge.

Case Studies and Learning Outcomes

Empirical comparisons between rule-based and machine learning systems in log analysis indicate that while ML models excel at identifying subtle and previously unknown patterns, rule-based systems remain indispensable in:

- Environments with constrained resources (e.g., IoT devices, edge computing).
- Initial event triaging and filtering at high ingestion rates.
- Use cases requiring traceable and deterministic decisions, such as financial systems and healthcare infrastructure.

Our literature analysis of CrowdSec confirmed the effectiveness of scenario-based rule definitions tailored to specific formats, particularly in detecting high-frequency attacks like brute force, as well as lateral movements when rules are properly correlated.

In conclusion, rule-based and decision-based techniques form the backbone of many practical SIEM solutions, especially when integrated into multi-layered detection frameworks that balance speed, clarity, and adaptability.

2.10 Security Information and Event Management Systems

Security Information and Event Management (SIEM) systems are a foundational component of modern cybersecurity architectures, designed to provide real-time analysis, centralized logging, and automated responses to security events. A SIEM platform collects log data from across an organization's IT infrastructure—including servers, firewalls, operating systems, applications, and security devices—and aggregates this information into a unified system for analysis and monitoring.

The core functionality of a SIEM system lies in its ability to perform correlation and detection of potential threats through two primary mechanisms: **log management** and **event correlation**. Log management involves the systematic collection, parsing, storage, and retrieval of log data, ensuring auditability

and enabling forensic analysis. Event correlation, on the other hand, utilizes predefined rules or machine learning models to detect patterns and relationships among different log entries that may indicate malicious or anomalous activity.

SIEM systems typically support features such as:

- **Real-time monitoring and alerting:** Identifying and notifying analysts about suspicious behaviors or violations of policy.
- **Threat detection and response:** Applying correlation rules or machine learning to detect intrusion attempts, lateral movements, and privilege escalations.
- **Data normalization and aggregation:** Standardizing heterogeneous log data into a common schema to allow unified querying and analysis.
- **Compliance reporting:** Generating reports to satisfy regulatory requirements such as GDPR, HIPAA, or PCI-DSS.
- **Historical data analysis:** Performing retrospective investigations to understand past attacks or potential vulnerabilities.

The importance of SIEM systems has grown significantly due to the increasing complexity and scale of cyberattacks. They empower security analysts to shift from reactive to proactive threat management by providing visibility into system behaviors across organizational boundaries. Modern SIEM platforms are increasingly integrating with threat intelligence feeds, anomaly detection algorithms, and SOAR (Security Orchestration, Automation, and Response) capabilities to improve detection accuracy and incident response times.

Relationship with Log Analysis

At the heart of every SIEM system is **log analysis**. Logs generated by servers, operating systems, applications, firewalls, routers, and intrusion detection systems provide a granular, time-stamped account of system behavior. SIEM platforms ingest and analyze this data to identify patterns that may signify unauthorized access, brute-force attempts, malware activity, privilege escalations, and other anomalies.

SIEM systems perform both **retrospective** and **real-time** log analysis:

- **Retrospective Analysis:** Involves parsing historical logs to understand the timeline and scope of a breach, often used during forensic investigations and audits.
- **Real-Time Analysis:** Enables continuous monitoring and immediate detection of suspicious events, triggering alerts and sometimes automated responses.

The effectiveness of a SIEM depends on its ability to normalize diverse log formats, correlate events across sources, and apply intelligent analytics to derive meaningful insights.

In the context of our project, SIEM systems serve as the theoretical and architectural backbone. The rule-based detection module, machine learning-backed

anomaly detection, and log correlation functionalities mirror the core principles found in industry-grade SIEM solutions like Splunk, IBM QRadar, and Elastic SIEM. Our implementation aims to distill these functionalities into a simplified but effective academic prototype that demonstrates core detection capabilities, log analytics, and user alerting through intuitive interfaces.

2.11 Technology Stacks in SIEM Systems

Security Information and Event Management (SIEM) systems vary significantly in their architecture and technology stacks, depending on their intended use-case, scalability requirements, and real-time performance capabilities. Traditional and commercial SIEM solutions such as Splunk, IBM QRadar, and ArcSight follow tightly integrated and often proprietary designs, while open-source systems like the ELK Stack and CrowdSec offer modular, flexible approaches better suited for community-based development.

The **ELK Stack** — consisting of Elasticsearch, Logstash, and Kibana — is one of the most widely adopted open-source platforms for log collection, indexing, and visualization. Elasticsearch serves as the search and analytics engine, Logstash performs log parsing and enrichment, and Kibana provides dashboard visualizations. While powerful and scalable, the ELK Stack often requires significant setup and computational resources, which may pose challenges for academic or student-led projects.

CrowdSec, on the other hand, is a modern, open-source intrusion prevention system that emphasizes collaborative security. It uses log parsing with YAML-based detection rules and supports decision trees for threat classification. CrowdSec inspired our decision to pursue rule-based log parsing techniques as an efficient and interpretable alternative to machine learning models, especially given the limitations in computational power and dataset availability often faced by student developers.

Commercial platforms like **Splunk** and **QRadar** utilize proprietary ingestion engines, complex rule engines, and high-performance backends. These systems are feature-rich but typically require licensing fees and access to high-volume log data — factors that limit their accessibility for research and educational experimentation.

In contrast to these large-scale systems, our stack emphasizes modularity and simplicity without compromising flexibility:

- **FastAPI (Python):** FastAPI is a modern, asynchronous web framework for Python that supports high-performance API development with automatic validation and documentation. Unlike traditional WSGI-based frameworks such as Flask or Django, FastAPI uses ASGI (Asynchronous Server Gateway Interface), allowing non-blocking request handling. This becomes especially valuable in real-time applications like SIEMs, where multiple logs are ingested simultaneously. In contrast to heavier, enterprise-level ingestion services (e.g., Logstash), FastAPI provides a lightweight alternative that is easier to develop, test, and deploy—making it particularly well-suited for academic environments where rapid prototyping and clarity of

code are essential.

- **ReactJS:** ReactJS, a JavaScript library developed by Facebook, is leveraged to build a dynamic and responsive frontend for the system. React’s component-based architecture enables clean UI separation and reusable logic, which simplifies the process of building complex dashboards for visualizing system status, anomalies, and rule matches. In industry settings, Kibana (as used in the ELK Stack) serves a similar purpose for visualization, but it is tightly coupled with Elasticsearch and less customizable for non-ELK data sources. ReactJS, being framework-agnostic and open-ended, offers more flexibility for integrating various APIs and crafting custom visual experiences.
- **Supabase (PostgreSQL):** Supabase is an open-source alternative to Firebase that sits on top of PostgreSQL. It provides built-in authentication, real-time subscriptions, and a RESTful API. PostgreSQL, as a robust and relational SQL database, offers strong support for transactions, ACID compliance, and complex queries — all of which are crucial for maintaining log integrity, enforcing relational constraints (e.g., user-device-log mapping), and querying threat intelligence across time and user profiles. Compared to Elasticsearch, which is optimized for full-text search but not for relational data, PostgreSQL provides greater reliability for structured storage in security systems.
- **MongoDB (NoSQL):** MongoDB is employed to simulate short-term memory and preserve contextual event data. Unlike relational databases that enforce rigid schema, MongoDB’s document-oriented structure is more flexible for storing time-series sequences, failed login attempts, or dynamic IP threat states. For instance, to detect brute force attempts or delayed DDoS patterns, the system needs to store evolving state data (e.g., IP access counts, timestamps, failure history), which is easier to manage in a NoSQL database. MongoDB’s ability to persist temporary state even across process restarts mimics the behavior of RNN-like memory systems without requiring actual deep learning models. This hybrid approach blends rule-based logic with memory persistence—often termed as “context-aware rules”—a technique also supported in commercial tools like QRadar with its flow processors.

2.12 Comparative Studies and Learning Outcomes

Comparative studies between rule-based systems and machine learning (ML) models in log analysis reveal significant trade-offs between interpretability, resource efficiency, and detection capabilities [20]. Rule-based approaches, such as those implemented in CrowdSec and OSSEC, are praised for their deterministic behavior and simplicity, allowing security analysts to quickly design and audit detection logic without the need for complex training pipelines [7]. These systems use structured patterns, thresholds, and correlation rules that are both human-readable and easily updated.

On the other hand, ML-based models—especially supervised learning algorithms—have

demonstrated higher effectiveness in identifying complex or previously unseen attack patterns, particularly in large-scale or noisy environments [21]. Deep learning architectures, such as LSTMs and autoencoders, have further improved anomaly detection by modeling long-range dependencies and subtle correlations in log streams [22]. However, these benefits come with steep costs: ML models demand significant computational resources, well-labeled datasets, and continuous retraining to maintain performance over time.

From an industry perspective, hybrid solutions are increasingly adopted. For example, platforms like Splunk and Elastic integrate rule engines with anomaly detection modules to provide tiered detection pipelines. In such setups, rule-based logic is often used for initial filtering or to capture well-known threats, while more advanced models are reserved for deep behavioral analysis [23]. This approach not only improves scalability but also ensures a balance between interpretability and adaptability.

However, despite their growing popularity, ML-based techniques face substantial barriers in real-world deployment, especially for smaller organizations:

- **Lack of labeled and clean datasets:** Quality training data is often unavailable or proprietary.
- **Complexity and maintenance:** ML pipelines require domain expertise, tuning, and lifecycle management.
- **Opaque decision-making:** Many ML models act as "black boxes", making it difficult to trace why an alert was triggered.
- **Gatekeeping in SIEM tools:** Many commercial SIEM platforms are costly or restricted, limiting experimentation or modification by small teams.

Given these challenges, our project adopts a pragmatic approach: we integrate a lightweight rule-based detection framework with a background scoring mechanism that mimics the adaptive behavior of ML models without requiring full-scale training. This algorithm tracks contextual attributes like IP behavior frequency and user activity history to enhance detection logic, while still maintaining transparency.

Our implementation, inspired by CrowdSec’s YAML-based rule architecture and Sigma rule specifications, offers the advantages of fast deployment, deterministic logic, and scalability across resource-constrained environments. This design ensures that the system remains explainable, performant, and extensible—while aligning with real-world operational constraints.

In conclusion, although ML methods hold promise in uncovering complex attack vectors, our literature review and empirical considerations affirm that a rule-based-first strategy, supplemented by contextual logic, is the most viable choice for our use case.

Chapter 3

Methodology

The methodology of this project follows a structured process to develop an efficient and practical SIEM system. Initially, we explored advanced machine learning techniques but later adopted a more manageable and scalable approach using log parsing and decision tree classification. The following sections outline the steps taken in the development of the Log Parser and Classifier Tree.

3.1 Initial Approach: BERT-based ML Model

Our first approach was to use BERT (Bidirectional Encoder Representations from Transformers), a powerful pre-trained model for Natural Language Processing (NLP), for log classification. BERT has been proven to perform well in tasks like text classification and entity recognition. So, LogBERT, a derivative of the BERT Model was to be used, which led us to believe it could be applied to classify logs into "Anomalous" or "Normal" categories.

The process included the following steps:

Data Preprocessing:

Logs were tokenized and preprocessed to convert them into a format suitable for input into BERT. We used various sequences to provide the BERT Model an understanding of the factors that it should look for to classify the logs as Anomaly. After that, we padded the sequences to provide the data with a proper format for the BERT Model to train on. We also randomly masked the data to differentiate between actual and padded tokens.

Log Parser

Our initial approach was to develop a Log Parser that would extract relevant features such as time-based Grouping, identify critical log entries within the mentioned specific time window and within the same day from logs, and many more based on the sequences provided so that the BERT Model can train on those and provide us the desired results.

Model Training:

We attempted to fine-tune a pre-trained BERT model using datasets of logs[24] after splitting the dataset into train and testing sets. This training process required significant computational resources, as fine-tuning BERT requires powerful hardware, which became impractical for our project.

Challenges:

During this stage, we faced several issues, including long training times, high computational costs, and difficulties in obtaining a proper result from the model. The complexity of tuning a BERT model for log analysis was overwhelming given the project’s scope, leading us to reconsider our approach.

3.2 Revised Approach: Log Parsing and Classification

After realizing the complexity of the initial approach, we revisited the project design by studying industry practices, particularly those shared by CrowdSec, an open-source cybersecurity platform. From their documentation, we found that many security tools rely on keyword-based log parsing and rule-based classifiers, to efficiently identify and classify logs.

Based on this insight, we decided to pivot to a simpler, yet effective, method involving a Log Parser and a Classifier Tree.

Log Parser

Our revised approach was to develop a Log Parser that would extract relevant features from logs based on predefined keywords or patterns. The parser performs the following tasks:

- **Log Extraction:** The parser reads log files and identifies critical log entries, such as timestamps, source IPs, actions, and error codes.
- **Keyword Identification:** It scans each log entry for specific keywords or patterns that are indicative of normal or anomalous behavior. These keywords might include specific phrases like “failed login attempt” or “unauthorized access.”

Classifier Tree

The Log Parser only allows the logs with the specified keywords to move forward to the classifier tree that can be used for classification.

- **Rules:** Various filters that are necessary for classifying whether the logs are something anomalous or normal are mentioned as rules, which help us separate the normal or non-anomalous logs from the anomalous ones.
- **Classification:** We used a classifier tree to classify the logs into two categories: Normal or Anomalous. The Classifier model makes predictions based on a series of decisions made.

This methodology, inspired by CrowdSec’s approach and combined with the simplicity and effectiveness of classification trees, allows for efficient log classification with minimal computational overhead, providing us with a very fast and reliable system for classifying threats.

Probabilistic Scoring and Short-Term Memory Queue

To address the limitations of purely rule-based systems, we introduced a lightweight memory mechanism that simulates context learning through probabilistic scoring and temporal retention of suspicious IPs. This module operates in tandem with the rule-based classifier to refine detection over time.

- **Anomaly Probability Calculation:** For every 1000-log batch, we compute a probability score for each IP based on how often it appeared in anomalous logs within that batch. The score for an IP is given by:

$$P(\text{anomalous} \mid \text{IP}) = \frac{\text{Number of times IP was anomalous}}{\text{Total logs in batch}}$$

A threshold (e.g., 0.01) is used to determine which IPs are considered significantly suspicious for further attention.

- **TTL-Based Short-Term Memory Queue:** Each IP that exceeds the anomaly threshold is added to a short-term memory queue with a Time-To-Live (TTL) counter. The TTL value is decremented for each new batch where the IP does not appear as suspicious. If the TTL reaches zero, the IP is evicted from the queue. This simulates a decaying memory, allowing the system to remember persistent offenders while forgetting benign or temporary anomalies.
- **Persistent Context Awareness:** The queue persists across batches and maintains state. This means that IPs with sporadic but repeated anomalous behavior are retained and monitored across time, even if they are not flagged in every batch. This temporal retention mimics a learning mechanism, enhancing our ability to detect low-frequency but consistent threats.
- **Real-Time Scoring Output:** At each batch iteration, the system outputs the current state of the memory queue, including the IP address, its latest anomaly score, and remaining TTL. This provides a real-time, interpretable view of currently suspicious entities.

Chapter 4

Implementation Steps and Details

4.1 Synthetic Log Generation Methodology

To effectively test our SIEM system under conditions mimicking real-world usage, we required a large volume of event logs. However, our original dataset consisted of only 2,000 Linux server logs. To overcome this limitation while maintaining data realism, we developed a synthetic log generation process inspired by principles from time-series data augmentation, bootstrapping, and event simulation. Below we outline and explain the components of this methodology.

Temporal Extension Through Timestamp Incrementation

A defining characteristic of server logs is their time-ordered nature. To extend the dataset temporally and emulate long-term server operation, we incremented the timestamp of each synthesized log entry in a controlled, realistic manner. The time increments were based on observed inter-log intervals in the original dataset, thus preserving the typical cadence of system activity.

This approach is analogous to techniques in time-series data augmentation, such as time warping or window slicing, where temporal continuity is preserved to maintain sequence-dependent patterns. It ensures that logs reflect both high-frequency and idle periods, making the dataset suitable for testing temporal analytics and detecting long-term trends or anomalies in the SIEM system.

Preservation of Source IPs and Log Templates

To simulate a consistent server environment, all generated logs retained the original IP addresses and message templates. In real-world systems, servers often interact with a stable set of clients or services. By maintaining these entities, we preserve structural consistency, enabling detection systems to learn from behavioral baselines.

This mirrors data generation techniques that rely on fixed categorical anchors—such as label conditioning in synthetic image generation (e.g., GANs (Generative Adversarial Networks) conditioned on class labels)—to ensure that synthetic data

remains contextually grounded. Retaining original log structures ensures the synthetic data remains within the same operational and semantic domain, which is critical for evaluating SIEM pattern recognition or signature-based alerting.

Random Sampling With Dynamic Seeding

Each generation cycle sampled 100 logs randomly from the cumulative dataset. To avoid deterministic repetition, we altered the random seed in each iteration. This introduces variability and ensures a high entropy dataset, which is essential for stress-testing anomaly detection algorithms.

This process draws parallels to bootstrapping methods in statistical resampling, where repeated sampling with randomness is used to estimate population properties. In our case, it introduces diversity into the synthetic logs, improving the SIEM system’s exposure to variations without straying too far from realistic scenarios.

Temporal Correlation Simulation

In real-world logs, events often occur in sequences or bursts—such as a login attempt followed by command executions or privilege escalations. To simulate this, we introduced a 10% probability that five consecutive logs from the original dataset would be copied as a block into the new data. This preserves the chronological integrity and narrative of short-term correlated activities.

This approach reflects ideas from Markov-based sequence generation and time-series bootstrapping, where short subsequences are reused to maintain event dependencies. By doing so, we enable the synthetic dataset to retain causal relationships and activity bursts, crucial for validating SIEM capabilities like rule chaining, alert correlation, and incident reconstruction.

Recursive Dataset Growth

Each iteration of log generation fed on the most current version of the dataset—including previously synthesized entries—creating a recursive growth model. This mechanism allows the dataset to evolve with increasing complexity, simulating a server running continuously and accumulating logs over months or years.

This recursive expansion is conceptually similar to progressive GANs or reinforcement learning environments where feedback influences subsequent outputs. It enables emergent behaviors or rare combinations of logs to arise organically over many iterations, increasing the dataset’s richness and helping the SIEM system encounter nuanced or unexpected scenarios.

Parameter Modularity

All parameters—such as the number of logs per iteration, the percentage chance of block sampling, or the timestamp increment strategy—were kept modular. This tunability allows for tailored scenario testing, including simulating high-traffic periods, DoS attacks, or long-term dormant states.

Such modularity reflects best practices in synthetic data frameworks, like the SDV (Synthetic Data Vault) or time-series simulation libraries, where domain-specific constraints and probabilities can be tuned. It provides flexibility to simulate various operational states and enables reproducibility of experiments by documenting parameter configurations.

Conclusion

By combining temporal realism, structural consistency, controlled randomness, and recursive growth, our synthetic log generation framework simulates extended server activity with high fidelity. Each component—rooted in data synthesis principles from time-series analysis, statistical resampling, and event simulation—plays a key role in generating data that is statistically valid, contextually realistic, and behaviorally rich.

4.2 Implemented Rule-Based Methods

In our project, we designed and implemented a series of rule-based log analysis techniques inspired by industry practices such as those found in CrowdSec. These rules aim to detect brute-force attempts, slow and distributed attacks, and other suspicious behaviors from SSH login logs. The system utilizes string matching, context preservation, and temporal checks to classify events as either normal or anomalous. Below, we outline each rule and its underlying rationale.

Rule 1: Detection of SSH Process Activity

We begin by checking if the log entry contains the string `"sshd"`. This string is commonly associated with the SSH daemon process on Linux systems and is used to identify events related to SSH connections. By filtering for `"sshd"`, we narrow our focus to logs generated by SSH activities, which are often the target of brute-force login attempts.

Rule 2: Detection of Authentication Failures

The next step is to inspect whether the log line contains the phrase `"authentication failure"`. This indicates a failed login attempt and is crucial for identifying potential unauthorized access. It helps flag any user or IP address repeatedly failing to authenticate via SSH.

Rule 3: Contextual Tracking of IP Addresses

To maintain historical context, we introduce a data structure (a list of lists or a dictionary) to track IP addresses that have triggered authentication failures. For each IP, we store:

- The IP address itself,
- The timestamp of its last failed attempt,
- The cumulative count of failures.

This enables stateful tracking of suspicious behavior over time, which is vital in log-based anomaly detection.

Rule 4: Time Window Correlation for Repeated Access

If an IP address is already known for previous failed attempts, we check whether the new failure occurred within a 10-second window from the last recorded attempt. This temporal proximity suggests a brute-force attack pattern and is a strong indicator of malicious activity.

Rule 5: Incremental Count for Detected IPs

If the 10-second window condition holds, the corresponding counter for that IP is incremented. This maintains a running count of failed attempts, which is later used for threshold-based decision making.

Rule 6: Threshold for Brute-Force Classification

If the number of recorded failures for a single IP exceeds 3 within the monitored time window, the activity is flagged as anomalous. This threshold is empirically chosen to balance between false positives (e.g., users mistyping passwords) and true brute-force behavior.

Rule 7: Detection of Distributed Slow Brute-Force Attempts

We check whether multiple different IP addresses have failed to authenticate more than 20 times within a 24-hour window. This identifies distributed or "low-and-slow" attacks, where the attacker avoids detection by spreading attempts over time and across IPs. If the condition is met, these entries are marked as anomalous.

Rule 8: Post-Failure Successful Login

In scenarios where a successful login is observed from an IP after multiple failures, we flag the event as a potential compromise. This pattern could indicate that an attacker eventually guessed the correct credentials, and immediate administrative investigation is required.

Rule 9: Multi-IP Access Within Tight Timeframe (DDoS Detection)

If several distinct IP addresses attempt SSH access and fail within a narrow 5-second window, this may signify a Distributed Denial-of-Service (DDoS) attack or automated scanning. All such IPs are flagged for anomaly.

Rule 10: System Exit Abnormalities

Finally, we inspect logs for system-level alerts using keywords such as "ALERT" and "exited abnormally". These messages typically indicate kernel crashes, unexpected shutdowns, or forced terminations. Any occurrence of these signals potential compromise or system failure and is marked as anomalous.

These rule-based methods form the core of our log analysis pipeline. Each rule is applied sequentially with context-aware tracking, allowing our system to flag suspicious behavior with minimal overhead while preserving explainability. This approach mimics some behavior of lightweight intrusion detection systems, ensuring both speed and reliability without the need for heavy machine learning infrastructure.

4.3 Short-Term Memory-Based Context Learning

Motivation and Design Rationale

Traditional rule-based intrusion detection methods offer high-speed evaluation and determinism, but struggle to generalize across complex contextual patterns, especially in the presence of evolving attack behaviors. To address this, we implemented a lightweight, memory-assisted scoring mechanism that acts as a short-term memory, capable of retaining suspicious IP addresses across multiple log batches. This module allows our SIEM system to gradually "learn" from patterns over time, much like a machine learning model, while avoiding the overhead and complexity of full-fledged training pipelines.

Batch-Based Log Processing

Our dataset consists of 20,000 system logs, which are processed in $n = 20$ batches of 1,000 logs each, simulating real-time ingestion of log data in chunks. Let $\mathcal{B}_t$ represent the t -th batch of logs.

$$\text{Logs} = \bigcup_{t=1}^n \mathcal{B}_t, \quad |\mathcal{B}_t| = 1000$$

Each batch is fed into both the rule-based detector and the short-term memory scoring module.

Probability-Based Anomaly Scoring

For every batch $\mathcal{B}_t$, we compute a probability score $P(\text{anomaly} \mid \text{IP})$ for each IP address that occurs in logs marked as anomalous by the rule-based system. The score is defined as:

$$P(\text{anomaly} \mid \text{IP}_i) = \frac{\text{Number of times IP}_i \text{ was anomalous in } \mathcal{B}_t}{|\mathcal{B}_t|}$$

This produces a score between $[0, 1]$ for each IP per batch, capturing how likely that IP is to exhibit anomalous behavior in the given context.

TTL-Based Suspicious IP Queue (Short-Term Memory)

To simulate a short-term memory, we maintain a persistent in-memory queue of suspicious IPs across batches. This is implemented using a TTL (Time-To-Live) mechanism, where each IP is retained for a fixed number of future batches (in our case, $\text{TTL} = 2$) unless it is refreshed by recurring anomalies.

Let:

- S_t be the set of IPs in batch $\mathcal{B}_t$ with $P \geq \theta$, where θ is a threshold (e.g., 0.01).

The update mechanism works as follows:

1. **Addition:** If $P(\text{anomaly} \mid \text{IP}_i) \geq \theta$, IP_i is added to S_t with $\text{TTL} = 2$.
2. **Decay:** For each existing IP in S_{t-1} , its TTL is decremented. If TTL reaches 0, the IP is removed.
3. **Persistence:** The queue is persistent across all batches, making it sensitive to patterns that evolve over time.

Visualizing and Tracking Memory State

To evaluate the system’s behavior, we plot the anomaly scores of all IPs in each batch, highlighting the suspicious ones retained in memory. We also log the complete queue state after every batch, showing IPs with their current TTL values. This enables traceability of when IPs are added, refreshed, or removed — a key feature mimicking the memory decay seen in neural systems.

Following are four graphs where anomaly scores vs IPs with a threshold of 0.01 for four batches are plotted sequentially

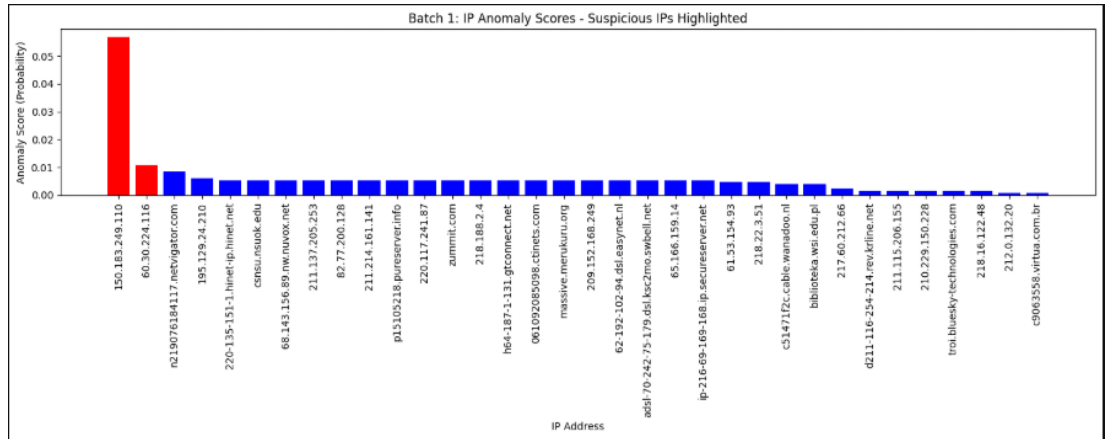


Figure 4.1: Score vs IP for batch 1

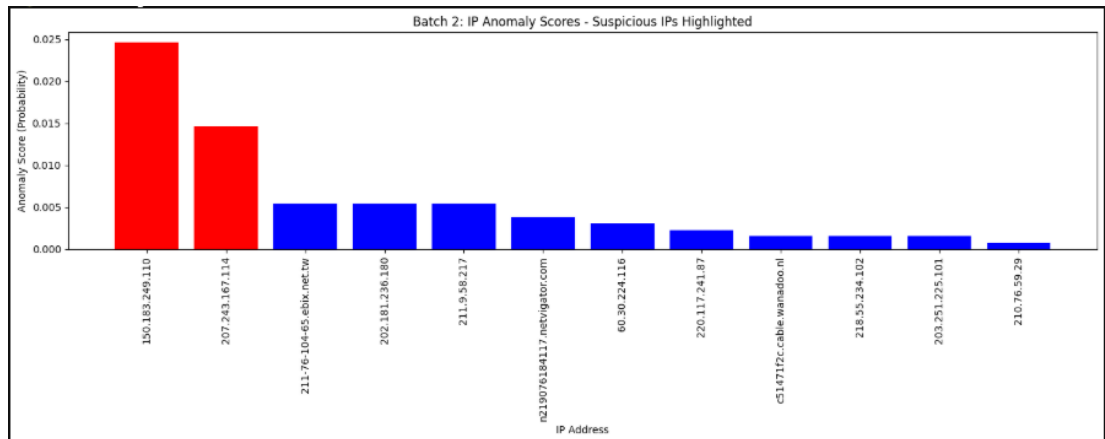


Figure 4.2: Score vs IP for batch 2

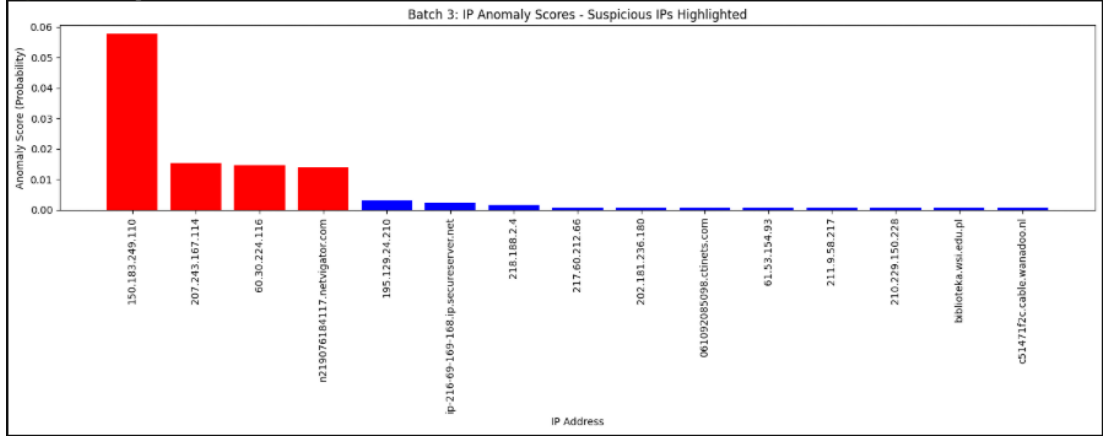


Figure 4.3: Score vs IP for batch 3

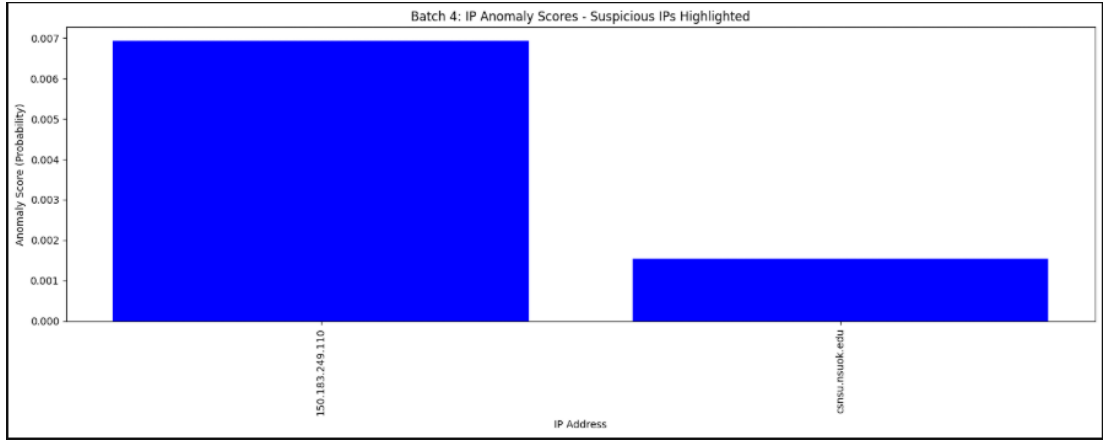


Figure 4.4: Score vs IP for batch 4

Short Term Memory Queue: The following text shows how the memory queue is dynamically updated and maintained across the four batches shown in the figures above. The "Added" indicates which IPs have been added to the S_t queue with a TTL of 2 and "Removed" indicates which ones have decayed (reached TTL 0) and have been removed from S_t .

Batch 1:

150.183.249.110 | Score: 0.0569 | TTL: 1
 60.30.224.116 | Score: 0.0108 | TTL: 1
 Added: 150.183.249.110, 60.30.224.116

Batch 2:

150.183.249.110 | Score: 0.0246 | TTL: 1
 207.243.167.114 | Score: 0.0146 | TTL: 1
 Added: 207.243.167.114
 Removed: 60.30.224.116

Batch 3:

```
150.183.249.110 | Score: 0.0577 | TTL: 1
207.243.167.114 | Score: 0.0154 | TTL: 1
60.30.224.116   | Score: 0.0146 | TTL: 1
n219076184117.netvigator.com | Score: 0.0138 | TTL: 1
Added: 60.30.224.116, n219076184117.netvigator.com
```

Batch 4:

```
Removed: 150.183.249.110, 207.243.167.114,
n219076184117.netvigator.com, 60.30.224.116
```

Approximate Context Learning Behavior

Although our method is not a machine learning model in the strict sense, it exhibits several behaviors that approximate key features of machine learning systems:

- **Temporal Pattern Retention:** The system is capable of remembering IPs that appeared suspicious in previous batches, even if they are not immediately anomalous in the current batch. This retention mimics a memory mechanism similar to recurrent neural networks, allowing the system to detect threats that unfold over longer temporal windows.
- **Probabilistic Reasoning:** Each IP is assigned a probability score that represents the likelihood of anomalous behavior within a given batch. This soft scoring allows us to differentiate between highly suspicious and marginally suspicious IPs, rather than relying on a binary classification. The probabilistic approach enables nuanced interpretation of threat levels.
- **Adaptive Behavior Over Time:** Since the memory queue evolves dynamically based on recent activity, the system adapts to changing threat landscapes. IPs are not permanently blacklisted; instead, they must continuously demonstrate suspicious behavior to remain in the queue, allowing for both persistence and decay.

Conclusion

By integrating this lightweight short-term memory mechanism into our SIEM system, we create a hybrid detection pipeline that combines the speed and clarity of rule-based methods with the contextual depth of temporal reasoning.

- **Efficiency and Interpretability:** Rule-based detection offers deterministic logic that is easy to audit and explain. The memory module builds on this by making decisions based on simple probability scores and TTL-based lifetimes, ensuring that all detection logic remains transparent and interpretable.
- **Scalable Contextualization:** The memory queue, which evolves batch-by-batch, acts as a scalable and efficient proxy for behavioral modeling. Without requiring large-scale model training, it enables the system to build a real-time understanding of persistent threats.

- **Near-ML Capabilities Without Complexity:** This method approximates learning behavior typically associated with ML systems—such as weighting recent evidence more heavily and retaining patterns across time without the complexity of model selection, parameter tuning, or training data imbalance issues.

4.4 Technology Stack

To ensure a modular, efficient, and scalable implementation of our Security Information and Event Management (SIEM) system, we have adopted a modern full-stack architecture. Each component of the stack has been chosen based on performance, ease of integration, and suitability for real-time log processing. Below, we detail the individual components and their roles in the system.

Backend: FastAPI (Python)

We use FastAPI as our backend framework for handling API requests and core logic execution. The core rule-based algorithm (RBA) and background short term memory queue algorithms work in tandem. The background algorithm stores the suspicious IP list in the database which can be accessed by the RBA working in the foreground. The RBA works and classifies logs and stores them in the database which in turn is accessed by the background algorithm in batches of approximately 1000 logs. The backend also handles API calls from the frontend and returns necessary information to be displayed in its dashboard.

Frontend: ReactJS

ReactJS is used for building the user interface of our application. The UI is a dashboard styled interface where the admin can monitor various information of flagged logs and suspicious IPs in the form of graphs and lists. We've tried to keep the UI clean and user friendly. The frontend periodically collects the required information from the backend and can also send requests to alert the respective devices' users through an email. The basic layout and design of the dashboard is showcased below.

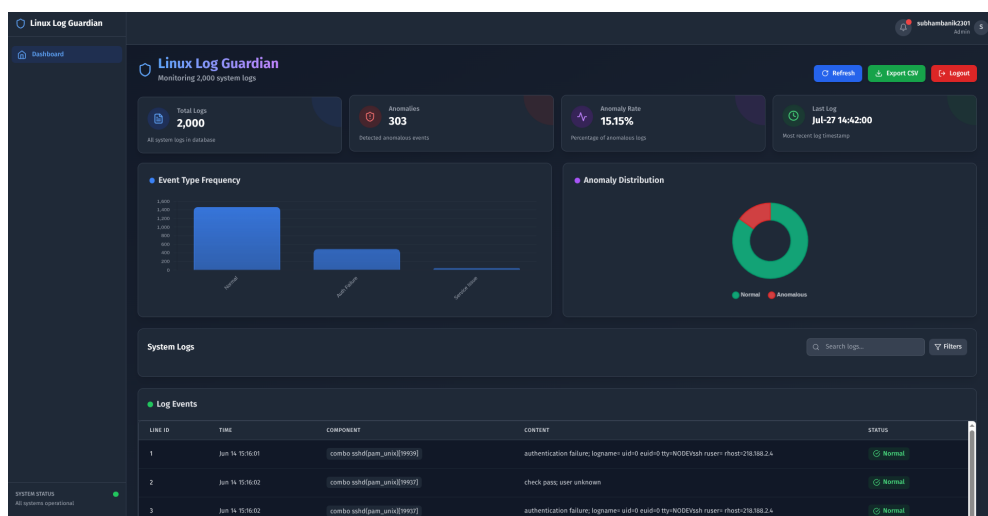


Figure 4.5: Admin Interface of the Dashboard

Database: Supabase (PostgreSQL)

Supabase serves as the primary SQL-based storage solution for our structured data. We have kept the structure quite simple with every log entry stored as its attributes in the log table. A separate table is kept for the suspicious IP list. The device table and the admin table have the details of the clients and their information that is useful for identifying a particular device and also to alert their user by sending an email.

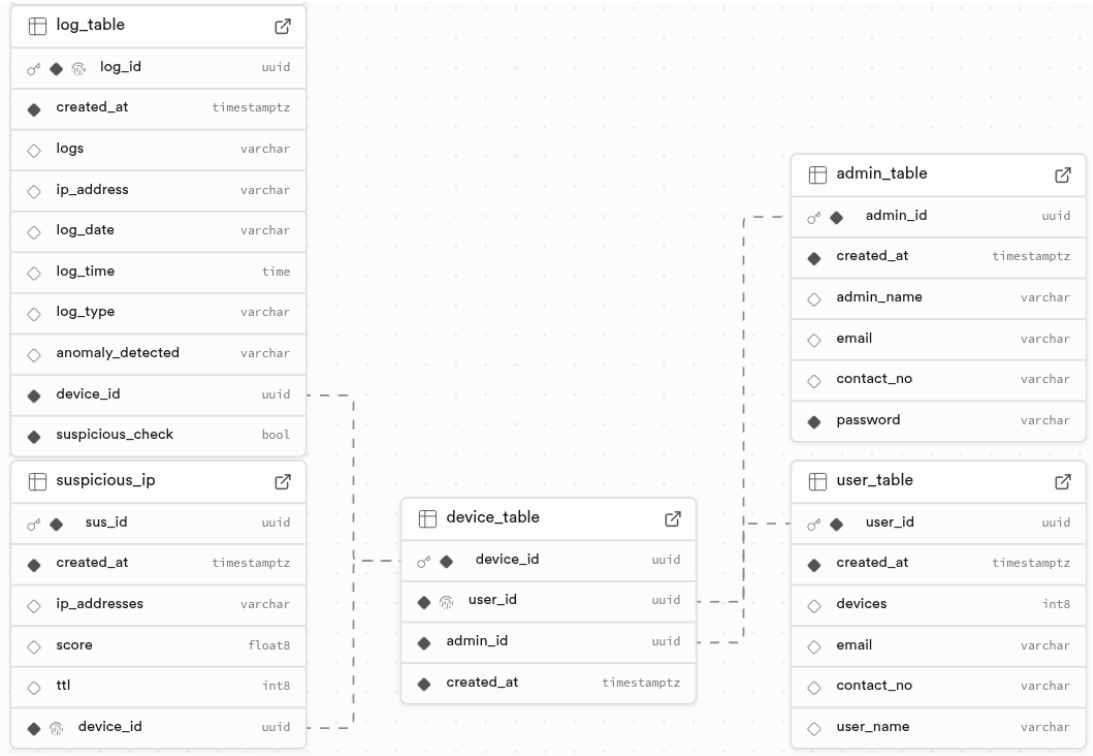


Figure 4.6: Basic structure of our database

Temporary Memory Store: MongoDB (NoSQL)

To simulate short-term memory and enable stateful rule-based evaluations (such as tracking past failed attempts from an IP address), we incorporate MongoDB as a temporary, volatile storage engine. This ensures contextual memory is preserved even in the case of a process restart. MongoDB's schema-less design and document storage model provide flexibility in managing complex state information, such as sequences of events per IP or user session.

Chapter 5

System Design and Workflow

This section presents the architectural overview and workflow of the designed Security Information and Event Management (SIEM) system. The architecture comprises a Linux client device (running on a server), a FastAPI-based backend, a ReactJS frontend, and a cloud-integrated database stack utilizing Supabase (PostgreSQL) and MongoDB (NoSQL). The communication between components is designed to be asynchronous, scalable, and efficient.

5.1 Linux Server Client

The system begins with a Linux-based client server that runs a bash script. This script is responsible for periodically extracting logs from the `‘/var/log/auth.log’` file every 10 seconds. The log data is then sent to the backend using an HTTP POST request.

On first contact, the server responds with a unique Device ID. For subsequent log uploads, the client sends logs based on the timestamp of the last sent log. This ensures that redundant data is not transferred and preserves bandwidth and storage efficiency.

5.2 Backend System (FastAPI)

The backend, built with Python’s FastAPI framework, is the core component responsible for log classification and anomaly detection. It consists of the two subsystems we have discussed previously:

Rule-Based Engine

This module implements the deterministic rule-based logic to classify log entries as anomalous or benign. The rules are designed to capture behavior indicative of brute-force attacks, DDoS attempts, or suspicious logins. Key features of this module include:

- Ingesting and parsing each log entry—extracting timestamp, process, and IP addresses.

- When an IP Address matches with any suspicious IP from the suspicious IP Address table, it is tagged as anomalous and the rule based check is bypassed.
- Parsing raw log data for process identification, keyword spotting (e.g., "authentication failure", "sshd"), and extracting IP addresses.
- Maintaining temporary session memory using MongoDB, which stores contextual data such as past failed login attempts, timestamps, and frequency counts per IP address.
- Checking rule conditions such as repeated authentication failures within 10 seconds, multiple IPs attempting access within 5 seconds (DDoS), or successful login after failures.
- If a known malicious IP or behavior pattern is detected, logs are classified as anomalous and stored in Supabase.

The Short-Term Memory Background Module

Complementing the rule-based system, this lightweight probabilistic module simulates machine learning behavior using context and short-term memory. As discussed previously it collects batches of log data from the database and based on a probability scoring adds or removes them from a TTL queue, which is in turn stored again in the database to be used by the rule based engine.

Alert and Notification System

Whenever a log sequence is detected as suspicious:

- The backend provides a functionality that can trigger a POST request to send an email alert to the system owner, informing them of potential brute-force or unauthorized access attempts. This is triggered only if the cybersecurity analyst wishes to do so after reviewing the data provided in the frontend.
- If the POST request is triggered, then the necessary information about the device and its user is collected from the database and the user is notified accordingly.

5.3 Frontend System (ReactJS)

The ReactJS-based frontend provides a user-friendly interface for the moderator or admin or cybersecurity analysts. It features:

- Real-time dashboards displaying suspicious IPs, detected anomalies, and system events.
- Graphs and visualizations showing login patterns, attack vectors, and alert trends.
- Buttons to manually trigger alerts or send emails to end users when unusual activity is observed.

5.4 Database Stack

- **Supabase (PostgreSQL)** stores all permanent data: classified logs, user metadata, and historical IP activity.
- **MongoDB (NoSQL)** is used as a temporary cache for context-related computations and in-memory rule processing. This design decision ensures that data is retained across short downtimes and power losses.

5.5 Communication Flow

This component orchestrates all interactions between clients, the backend, and the notification system:

- **Log Ingestion (POST):** The Linux client issues an HTTP POST every 10 s, sending new `/var/log/auth.log` entries to the FastAPI backend.
- **Dashboard Polling (GET):** The frontend dashboard periodically sends HTTP GET requests to the backend to retrieve the latest flagged logs and suspicious IPs for display.
- **Alert Delivery (POST):** Upon detecting an anomaly, based on the cyber-security analyst's review, the backend invokes a POST call to the email service (via FastAPI background task), triggering an email notification to the system owner.

5.6 Workflow Summary

The end-to-end operational flow of the SIEM system is outlined as follows:

1. **Device Setup and Log Extraction:** The owning party installs and configures a bash script on their Linux-based server(s). This script extracts logs periodically (every 10 seconds) from files such as `/var/log/auth.log`. On initial setup, the script communicates with the backend to register the device and receive a unique Device ID. For subsequent executions, it only sends newly generated logs based on the last sent timestamp.
2. **Log Transmission to Backend:** The client continuously sends POST requests to the backend containing new log entries. These transmissions are lightweight and designed to preserve bandwidth.
3. **Backend Parsing and Classification:** Upon receiving the logs, the backend system:
 - Parses each log entry to extract metadata such as IP addresses, timestamps, and authentication results.
 - Applies the rule-based decision engine to classify suspicious patterns (e.g., brute force attempts, DDoS indicators).
 - Temporarily caches session-related information in MongoDB to enable contextual decision-making.
 - Stores classified results and metadata permanently in Supabase.

- Optionally, applies a lightweight background probabilistic model to validate or supplement the rule-based decision.
4. **Frontend Interaction by Cybersecurity Specialist:** A cybersecurity specialist signs into the system via the ReactJS frontend, which is secured using Supabase Authentication (ensuring robust key management and session control). Once authenticated, the specialist can:
 - Access a real-time dashboard displaying logs, detected anomalies, and suspicious IP addresses.
 - Visualize trends and summaries using charts and data cards.
 - Filter, search, or drill down into specific events or devices.
 5. **Alert Escalation (Manual Notification):** If the cybersecurity analyst identifies a critical threat or highly suspicious activity from the dashboard, they may manually trigger a notification via a button on the frontend. This action issues a POST request to the backend, which in turn sends an automated email alert to the owning party of the affected server or device, warning them about potential compromise or brute-force activity.

This structured and modular workflow enables real-time monitoring, context-aware detection, and human-in-the-loop validation while ensuring scalability, resilience, and maintainability of the system.

5.7 Flowchart of the System Design

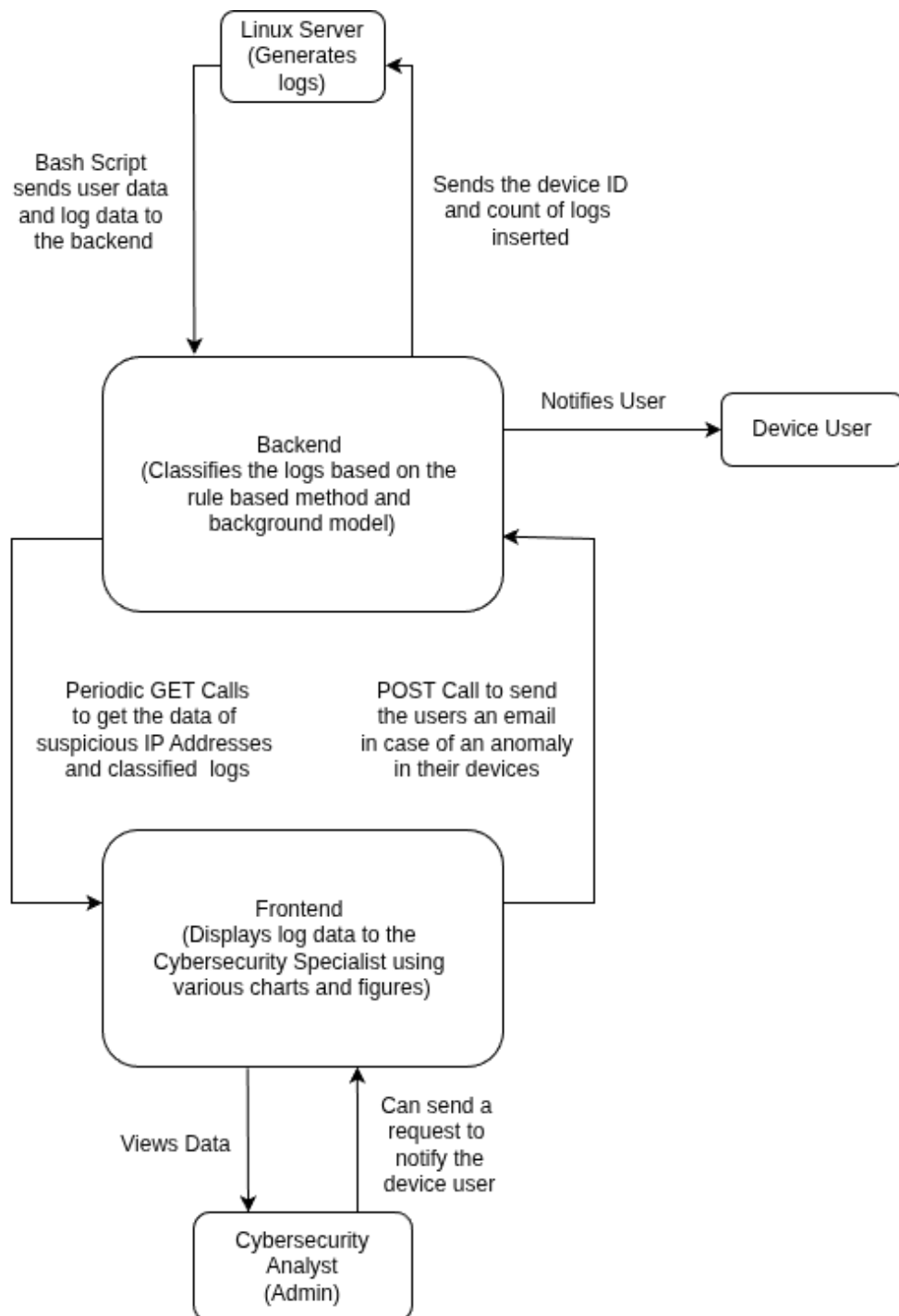


Figure 5.1: System Design and Workflow

Chapter 6

Future Prospects

The current SIEM system developed as part of this project presents a strong baseline for real-time log monitoring and anomaly detection using a rule-based and probabilistic hybrid approach. While it effectively handles basic threat detection scenarios, there are several future directions through which the system can evolve to compete with and complement existing enterprise-grade SIEM platforms.

Comparison with Existing SIEM Systems

Commercial SIEM platforms such as **Splunk**, **Elastic Stack (ELK)**, and **IBM QRadar** offer powerful capabilities, including full-text indexing, customizable dashboards, correlation rules, advanced anomaly detection, and integration with threat intelligence feeds. These systems are designed for high-throughput environments and support real-time alerting, forensic investigation, and compliance reporting.

In contrast, our system focuses on simplicity, interpretability, and extensibility within a smaller-scale academic context. Unlike traditional SIEMs, which require complex setups and high resource consumption, our system uses:

- A lightweight rule-based **Classifier Tree** for fast filtering.
- A memory-efficient **Short-Term Memory Queue** to simulate contextual awareness.
- A **probability scoring mechanism** to mimic anomaly trends across batches.

This minimalist design has shown that meaningful detection can still be achieved without heavy infrastructure or full ML pipelines, especially in resource-constrained or educational environments.

Proposed Improvements

To enhance the effectiveness and scalability of the system, several improvements can be pursued in future iterations:

- **Integration of Supervised Machine Learning:** While our current model mimics learning via heuristics, incorporating real ML models such as Ran-

dom Forest, Isolation Forest, or even lightweight LSTM-based sequence models could improve anomaly classification. These models can learn from labeled log histories and improve detection accuracy over time.

- **Temporal Correlation Engine:** Modern threats often involve low-frequency, coordinated activities across time. Implementing a proper correlation engine to analyze sequences of events rather than isolated logs can detect slow-burn attacks and lateral movement within a system.
- **Log Normalization and Parsing Framework:** Adopting structured log formats such as JSON or using log shippers like `Fluentd` or `Logstash` can enable better schema-based parsing and reduce misclassification due to inconsistent formats.
- **Threat Intelligence Integration:** The system can be connected to external threat intelligence sources (e.g., AbuseIPDB, VirusTotal APIs) to validate suspicious IPs or correlate events with known attack patterns.
- **Scalability Enhancements:** Using serverless technologies (e.g., AWS Lambda, Supabase functions) and message brokers (e.g., Kafka, RabbitMQ) can prepare the system to handle larger volumes of log data in a distributed and fault-tolerant manner.

Research Potential

From an academic perspective, this project opens up opportunities to explore the intersection of symbolic AI (rules, logic) and statistical learning (probabilistic scoring, ML models). It also encourages investigation into:

- Optimal TTL strategies for short-term memory decay models.
- Probabilistic threshold adaptation using dynamic baselines.
- Concept drift in log-based anomaly detection.
- Federated or collaborative learning across distributed SIEM nodes.

Our current SIEM system project demonstrates that a thoughtfully designed hybrid model can effectively approximate the intelligence of more complex systems while maintaining transparency and performance. By iteratively extending the system with modular improvements, it has the potential to grow into a robust and scalable tool. With continued development, integration, and testing, it can eventually reach near-enterprise standards and contribute meaningfully to open-source security monitoring efforts.

Chapter 7

Conclusion

The SIEM System developed as part of this project provides an effective and extensible framework for real-time log monitoring, analysis, and anomaly detection. Initially, the project aimed to implement a Machine Learning-based solution using models like BERT (LogBERT). However, after assessing the complexity and practicality within the scope of the project, we transitioned towards a rule-based and probabilistic approach that strikes a better balance between performance, interpretability, and feasibility.

The core of our system consists of a custom-built Log Parser that extracts meaningful features from raw log data using keyword and pattern matching. These parsed logs are then passed through a rule-based Classifier Tree, which applies domain-specific filters to categorize logs as either “Normal” or “Anomalous.” This method ensures fast, interpretable, and reliable log classification, which is crucial for real-time systems.

Recognizing that rule-based approaches, while efficient, often fail to capture broader behavioral context or temporal patterns, we extended our system with a novel lightweight probabilistic memory mechanism. By assigning anomaly probability scores to IP addresses based on their activity over 1000-log batches and maintaining a TTL-based memory queue, our system can identify IPs exhibiting low-frequency but consistent suspicious behavior over time. This memory mimics a decaying context window, simulating the benefits of Machine Learning models while maintaining simplicity and transparency. The modular nature of our implementation ensures that each component (parser, classifier, memory queue) can be independently tuned or replaced, allowing for future extensibility.

In the future, this system can be expanded further to integrate supervised or unsupervised ML models for richer anomaly detection, real-time alerting mechanisms, and persistent memory storage through external databases.

In conclusion, this SIEM system lays a strong and practical foundation for real-time security event analysis. It demonstrates how simple, interpretable techniques can be creatively combined to approach the effectiveness of more complex ML-based systems, while remaining lightweight, scalable, and easy to audit. The insights and architecture developed here offer a valuable stepping stone for more advanced security analytics systems in the future.

Bibliography

- [1] Wikipedia contributors. Security information and event management, 2024. Accessed: 2024-09-12.
- [2] IBM. Security information and event management (siem), 2023. Accessed: 2024-09-12.
- [3] Syed Abdul. Logbert: Log file anomaly detection using bert — an explainer, 2023. Accessed: 2024-11-02.
- [4] Helen Guo. Logbert: A log file anomaly detection framework using bert, 2023. Accessed: 2024-11-05.
- [5] CrowdSec Team. Crowdsec documentation. <https://docs.crowdsec.net/>, 2023.
- [6] CrowdSec. Parser:introduction, 2023. Accessed: 2024-11-16.
- [7] CrowdSec. Crowdsec documentation, 2023. <https://docs.crowdsec.net/>.
- [8] Fortinet. What is siem?, 2023. Accessed: 2024-09-12.
- [9] Noyan Tendikov, Leila Rzayeva, Bilal Saoud, Ibraheem Shayea, Marwan Hadri Azmi, Ali Myrzatay, and Mohammad Alnakhli. Security information event management data acquisition and analysis methods with machine learning principles. *Results in Engineering*, 22:102254, 2024.
- [10] David S. F. Jang. *Security Information and Event Management (SIEM) Implementation*. McGraw-Hill Osborne Media, 2010.
- [11] TechTarget. What is bert? definition from searchenterpriseai, 2023. Accessed: 2024-11-12.
- [12] Wikipedia contributors. Bert (language model), 2024. Accessed: 2024-11-12.
- [13] GeeksforGeeks. Explanation of bert model in nlp, 2023. Accessed: 2024-11-09.
- [14] Haixuan Guo, Shuhan Yuan, and Xintao Wu. Logbert: Log anomaly detection via bert. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2021.
- [15] Atomicorp Inc. Ossec documentation. <https://www.ossec.net/docs/>, 2023.

- [16] Sigma Project. Sigma rule specification. <https://github.com/SigmaHQ/sigma>, 2023.
- [17] Martin Roesch. Snort - lightweight intrusion detection for networks. *Proceedings of the 13th USENIX Conference on System Administration*, pages 229–238, 1999.
- [18] OISF. Suricata ids/ips/nsm engine. <https://suricata.io/>, 2023.
- [19] Sysdig, Inc. Falco: Cloud-native runtime security. <https://falco.org/>, 2023.
- [20] Juan Garcia and Alice Lee. Comparative analysis of anomaly detection techniques for security log data. *Journal of Cybersecurity Research*, 8(2):112–130, 2021.
- [21] Reza Sadoddin and Ali A Ghorbani. Anomaly detection in logs using unsupervised learning. *Security and Privacy*, 3(1):e98, 2020.
- [22] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298. ACM, 2017.
- [23] MITRE Corporation. Mitre att&ck framework, 2020. <https://attack.mitre.org/>.
- [24] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. Loghub: A large collection of system log datasets for ai-driven log analytics. In *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*, 2023.